

Auditoría portal

Introducción

Este reporte analiza el portal bajo dos lentes complementarios: el marco OWASP top 10, el cual clasifica las diez categorías de riesgo más críticas en aplicaciones web, y la ausencia de protección anti-bot, para la cual se propone Cloudfare Turnstile como solución. Los módulos auditados fueron Trámites, Consulta de adeudos, órdenes de pago, facturación, Mi perfil.

Hallazgos

CRÍTICOS

NETPAY_PRIVATE_KEY expuesta en logs de servidor

Campo	Detalle
Archivo	src/actions/ pagos.ts L925
Impacto	La clave privada de tokenización aparece en el stdout de Railway. Cualquier colaborador con acceso al panel de logs puede verla.
Fix	Eliminar console.log()
Área	FRONT

ActualizarMisDatos sin cabecera Authorization

Campo	Detalle
Archivo	src/actions/ mi-cuenta.ts L552
Impacto	El PATCH al API del SIR no lleva token — el recurso queda sin protección verificable desde el portal.
Fix	Agregar header Authorization con el token de sesión del usuario.
Área	FRONT

- OWASP 2025: A01 Broken Access Control

Monto de pago calculado en el cliente, sin revalidación server-side

Campo	Detalle
Archivo	PaymentControls.tsx L104–138 + pagos.ts L183–248
Impacto	Un atacante puede interceptar el payload de la Server Action y pagar un monto arbitrario. El servidor reenvía el amount del cliente al API de recaudación sin recalcularlo.
Fix	Recalcular el monto en generarOrdenDePago re-consultando los cargos al API de recaudación antes de reenviarlos.
Área	AMBOS — fix real es 100% BACK (pagos.ts); el frontend (PaymentControls.tsx) no requiere cambio de lógica, pero sí pruebas de regresión tras el fix.

- OWASP 2025:A06 Insecure Desing - A08 Integrity Failures

SDKs de NetPay y Banamex sin SRI y con URL editable desde el CMS

Campo	Detalle
Archivo	NetPayProvider.tsx L173 · BanamexCheckoutDialog.tsx L357
Impacto	Un admin CMS comprometido puede sustituir el SDK por un keylogger de tarjetas. Sin Content-Security-Policy que restrinja script-src. Es la categoría A03:2025 Supply Chain Failures.
Fix	Agregar atributo integrity (SRI hash) al <Script> de NetPay. Para Banamex, validar el dominio de scriptUrl contra una allowlist en código, no en el CMS.
Área	FRONT

- OWASP 2025 : A03 Software Supply Chain Failures

Sin rate limiting ni CAPTCHA en autenticación

Campo	Detalle
Archivo	src/actions/auth.ts L25 (solicitar OTP) · L114 (verificar OTP)
Impacto	Fuerza bruta sobre OTP de 6 dígitos sin bloqueo (1 M combinaciones). Enumeración de RFC/CURP vía respuesta 200 vs 204.
Fix	Integrar Cloudflare Turnstile en solicitarCodigoDeAcceso, obtenerAcceso y registrarCiudadano/Contribuyente. Rate limit en middleware.
Área	AMBOS — BACK: rate limit en middleware + verificación del token de Turnstile. FRONT: montar el widget de Turnstile en los 3 formularios.

- OWASP 2025:07 Authentication Failures

DobleInputForm (Servicios Registrales) con respuesta mockeada

Campo	Detalle
Archivo	src/components/frontend/pagos/formulario-pago/DobleInputForm.tsx L48
Impacto	El módulo nunca conecta al API real — siempre simula éxito, enmascarando que el endpoint real no está implementado
Fix	Implementar la llamada al endpoint real de Servicios Registrales.
Área	AMBOS — este es un módulo pendiente de desarrollo, no solo un bug. BACK: confirmar/exponer el endpoint real de Servicios Registrales. FRONT: quitar el mock y consumir el endpoint real. Ver Módulos Pendientes de Desarrollo.

- OWASP 2025: Mishandling of Exceptional Conditions

postRegistro loguea cuerpo completo con RFC, CURP, correo y contraseña

Campo	Detalle
Archivo	src/actions/auth.ts L224–225
Impacto	PII de nuevos registros expuesta en logs de Railway (dos líneas: L224 loguea la URL, L225 loguea el body completo).
Fix	Eliminar after(() => console.log(body)) en postPadronRegistro.
Área	FRONT

- OWASP 2025: Security Misconfiguration Protección de datos personales

ALTOS

Sin headers de seguridad HTTP en next.config.mjs

Ausentes: CSP, X-Frame-Options, HSTS, X-Content-Type-Options, Referrer-Policy.

Riesgo: Clickjacking, MIME sniffing, ataques de downgrade.

Área: FRONT

~80 console.log en producción exponiendo folios, URLs de gateway y datos de usuario PaymentControls.tsx concentra 18–20 instancias.

Área: FRONT

window.postMessage(response, '*') en NetPayProvider.tsx L54

Wildcard expone la respuesta de transacción a iframes de terceros en la misma sesión. Fix: usar process.env.NEXT_PUBLIC_APP_URL como targetOrigin.

Área: FRONT

getPortalSettings lee dos globals secuencialmente

Archivo: src/lib/portal-settings.ts L103-140

Fix: Promise.all([findGlobal('portal-settings'), findGlobal('payment-settings')]).

Reduce latencia del primer byte en un RTT completo en cada request.

Área: FRONT

N+1 en getMiExpedienteDocumentos

Archivo: src/actions/mi-cuenta.ts L658

1 fetch de lista + hasta 50 fetches individuales = 10-15 s en conexión lenta.

Fix: Promise.allSettled() o endpoint batch.

Área: FRONT, (en caso de que no se pueda un batch habilitar back)

expiryDate de órdenes hardcodeado a +3 días, ignorando diasInhabiles

La función calcularFechaExpiracionInteligente ya existe en pagos-formatter.ts.

Solo falta conectarla en buildOrdenPayload. Potencial IDOR.

Área: AMBOS

getMisVehiculos / getMisDeclaraciones usan fetchPublicJson (sin token)

Si el API no requiere autenticación propia, los datos son accesibles sin sesión.

Área: AMBOS, Front mandar verificación, back confirmarlo

Sin ErrorBoundary global en el layout raíz del frontend

Un error en /pagos, /facturacion o /mi-cuenta muestra la pantalla genérica de

Next.js sin contexto ni opción de recuperación.

Área: FRONT

Sin integración de plataforma de observabilidad (Sentry / Datadog)

Los errores no capturados se pierden. Sin alertas de SLA ni detección de ataques.

Área: AMBOS(SDK cliente + servidor)

padron_id y cargo_ids provienen del cliente sin validar ownership

Un usuario autenticado podría generar órdenes sobre cargos de terceros.
Área: AMBOS, Back validar que sea el mismo cliente, Front mandar el cliente

login_attempts en esquema de BD pero nunca leído ni escrito en código
El lockout de intentos fallidos está diseñado pero no implementado.
Área: FRONT

Sin log de auditoría de cambios en PaymentSettings (scriptId, claves)
Un admin podría modificar la URL del SDK de NetPay o Banamex sin dejar traza.
Área: FRONT

IDs de `billing.descuentos` no verificados contra los cargos ni contra el contribuyente.
Un atacante puede enviar descuentos de otro contribuyente o inexistentes
Área: BACK

MEDIOS

Descripcion	Área
clearTransactionData usa setItem(key, "") en vez de removeItem(key) — los datos de transacción persisten como string vacío. hasPendingTransaction evalúa "!!" = false correctamente, pero las claves no se eliminan del dispositivo.	FRONT
postFacturar lee data.error antes de verificar !response.ok — un error no-JSON enmascara el código HTTP original.	FRONT
registrarVisita falla silenciosamente — analytics de visitas completamente ciego.	FRONT
Sin indicador visual de reintentos en ConsultaAdeudoSection (máx. 3 intentos). El usuario ve spinner indefinido sin saber cuántos intentos quedan.	FRONT
revalidate = 3600 en /pagos es anulado por force-dynamic del layout raíz — directiva letra muerta.	FRONT
payload-types.ts desincronizado — cast any-like en portal-settings.ts. Fix: pnpm generate:types.	FRONT
ENTIDAD_ID = 1 hardcodeado en ConsultaAdeudoSection.tsx L30 y facturacion.ts L37. Nota: mi-cuenta.ts ya usa process.env.APP_ID_ENTIDAD ?? 1. Centralizar en PortalSettings o variable de entorno.	AMBOS
paymentError:v1 en localStorage persiste indefinidamente sin TTL.	FRONT
Sin rotación de cookie tras autenticación exitosa (session fixation parcial).	FRONT

PredioInputForm con búsqueda incompleta (TODO activo en producción).	FRONT
crearSesionBanamex retorna null en error; confirmarPagoBanamex retorna {success:false} — inconsistente con confirmarPagoConvenioBanamex que lanza ServiceRequestError. Estandarizar todo banamex.ts a ServiceRequestError.	FRONT
handleReset en FacturacionLookup no limpia emailInput ni emailLoadingId — puede dejar el UI en estado inconsistente.	FRONT
buttonVariants y cn duplicados localmente en FacturacionLookup.tsx. Importar desde @/components/ui/button y @/lib/utls.	FRONT
force-dynamic en layout raíz re-renderiza incluso páginas completamente estáticas (/tramites, /oficinas) en cada request.	FRONT
Logging de datos de usuario en mi-cuenta.ts es más amplio de lo esperado: no es por acción individual sino vía el helper fetchJson (L46, L60) compartido por todas las llamadas autenticadas. Cualquier respuesta con RFC, CURP o correo pasa por ese logger	FRONT

Exposición del portal

OWASP 2025

CATEGORÍA	NIVEL	HALLAZGOS EN EL PORTAL
A01 · Broken Access Control	CRÍTICO	PUT sin Authorization, IDOR potencial en cargos de terceros, endpoints públicos en vehículos/declaraciones
A02 · Security Misconfiguration	CRÍTICO	Sin headers HTTP, clave privada NetPay en logs, PII en stdout de Railway
A03 · Supply Chain Failures	CRÍTICO	SDKs de NetPay y Banamex sin SRI, versión no fija, sin auditoría en CI
A04 · Cryptographic Failures	ALTO	postMessage wildcard en dos instancias, localStorage sin TTL
A05 · Injection	OK	URLSearchParams + Zod + sin dangerouslySetInnerHTML — sin hallazgos
A06 · Insecure Design	CRÍTICO	Monto de pago calculado en cliente, sin pago parcial de adeudos, OTP sin lockout

A07 · Authentication Failures	CRÍTICO	Sin rate limit en OTP (1M combinaciones), login_attempts diseñado pero no implementado
A08 · Integrity Failures	CRÍTICO	SDKs sin SRI, monto de pago sin revalidación server-side
A09 · Logging & Alerting	ALTO	Sin logging estructurado, sin alertas de ataque, sin Sentry/Datadog
A10 · Mishandling Exceptions	CRÍTICO	Mock en producción (DobleInputForm), errores silenciosos en Banamex

Ataque por bots

PRIORIDAD	ENDPOINT / FORMULARIO	UBICACIÓN	RIESGO	OWASP	Área
CRÍTICO	solicitarCodigoDeAcceso	auth.ts:L25	Enumeración masiva de RFC/CURP/correos – respuesta 204 vs 200 actúa como oráculo	A07	AMBOS
CRÍTICO	obtenerAcceso	auth.ts:L114	Fuerza bruta sobre OTP de 6 dígitos – 1,000,000 combinaciones sin bloqueo	A07	AMBOS
CRÍTICO	registrarCiudadano / registrarContribuyente	auth.ts:L291, L311	Registro masivo de cuentas fake que contamina el padrón fiscal del SIR	A07	AMBOS
ALTO	consultarCargos	[padron]/page.tsx	Scraping masivo del estado fiscal enumerando IDs/RFC sin autenticación	A01	AMBOS
ALTO	buscarPorPadronId	catalogos-adeudos.ts	Scraping de padrones completos vía búsqueda pública sin throttling	A01	AMBOS
MEDIO	fetchRecibosExternos / postFacturar	facturacion.ts	Fuerza bruta de folios fiscales para validar RFC/factura	A07	AMBOS

ESTADO DE FUNCIONALIDADES

ESTADO	MÓDULO	NOTAS
FUNCIONAL	Trámites (listado, detalle, citas, PDF)	—
FUNCIONAL	Pago NetPay, BBVA, Santander, Banamex	—
FUNCIONAL	Órdenes de Pago	Caveat: vencimiento ignora días inhábiles (ALTO-07)
FUNCIONAL	Facturación (consulta, PDF, email)	Caveats: sin validación de email, error handling inconsistente
FUNCIONAL	Mi Cuenta — Mis Datos	Auth del PUT cuestionable (CRIT-02)
ROTO	Consulta de Adeudos — DobleInput (Servicios Registrales)	Siempre retorna mock hardcodeado — nunca conecta al API real (CRIT-06)
ROTO	Analytics de visitas (registrarVisita)	Falla silenciosamente — sin retry ni log de errores
INCOMPLETO	Consulta de Adeudos — PredioInput	TODO activo en producción, sin implementar
INCOMPLETO	Selección parcial de cargos	No existe UI — siempre se seleccionan todos los cargos
INCOMPLETO	Normateca	Endpoint no conectado — módulo sin datos reales
AUTH INCIERTA	Mi Cuenta — Mis Vehículos / Mis Declaraciones	Usan <code>fetchPublicJson</code> sin token (ALTO-08)
AUTH INCIERTA	Mi Cuenta — Expediente	Funcional pero con overhead de hasta 50 conexiones concurrentes (ALTO-05)

Accesibilidad

ID	ELEMENTO	PROBLEMA	NIVEL
AC-01	ServiceErrorState	Usa <code>role="status"</code> (polite). Para errores que requieren acción, debería ser <code>role="alert"</code> (assertive).	AA
AC-02	FacturacionLookup email input	Sin <code>aria-label</code> explícito — depende del placeholder, no accesible para lectores de pantalla.	AA
AC-08	C	No auditado — requiere herramienta de análisis de color. Presets personalizados pueden violar WCAG AA 4.5:1.	Pendiente

Migración de lógica de importes

Toda la lógica del cálculo vive actualmente en el frontend, esto expone la manipulación externa. La migración mueve esa lógica al backend, donde el cliente solo envía intenciones (cargos a pagar y descuentos) y el servidor valida, calcula y genera la referencia.

Punto Vulnerable	Riesgo concreto
billing.importe lo calcula el frontend y lo manda	Un cliente puede modificar el monto antes del POST y pagar menos
billing.descuentos es una lista de IDs elegida por el frontend	Un cliente puede inyectar IDs de descuentos que no le corresponden
La lógica de solo_aplica_el_descuento_mayor está en JS	Se puede omitir y mandar múltiples descuentos que debería excluir uno al otro
aplicaDescuentoPorPeriodo lo resuelve el frontend	Se puede activar sin tener todos los periodos requeridos y obtener descuentos indebidos
Los filtros estados_globales, cancelado, saldado los aplica el fronten	Se pueden saltar y mandar descuentos inactivos o ya soldados
billing.cargos es elegida por el frontend	Se pueden omitir cargos y generar una referencia parcial sin autorización del servidor

Recalcular siempre en el servidor, ignorar lo que manda el cliente

- adeudo_total por cargo el backend no debe confiar en el importe recibido, debe recalcular sumando adeudo_base + recargos + actualizaciones + gastos - descuentos válidos
- validación de billing.descuentos, cada ID en la lista debe pertenecer a uno de los cargos enviados y pasar todos los filtros de elegibilidad. Rechazar la petición si alguno no es válido.
- Importe final de la referencia, el backend calcula el total a partir de sus propio adeudo_total por cargo, no del importe que llegó en el payload
- Mover al servidor la fecha de expiración, actualmente el frontend decide cuándo vence la referencia, el servidor debe calcularla en base del tipo del padrón y el calendario de días hábiles, sin aceptar la fecha del cliente.
- la lógica de multas y accesorios normales, la separación entre recargos/gastos y multas afectan a los descuentos aplicables y debe validarse en el servidor

Contrato propuesto para el nuevo endpoint

El frontend solo envía una intención, nunca montos calculados

POST /recaudacion/referencias-pago-netpay-public

```
{
  billing: {
    cargos: [id, id, ...], // IDs de cargos que el contribuyente quiere pagar
    descuentos: [id, id, ...], // IDs de descuentos que el contribuyente quiere aplicar
    padron_id: number,
    tipo_de_padron: number,
    contribuyente: { id, ... },
    referencia_normal: boolean,
    convenio: number | null,
  }
  // SIN importe, SIN expiryDate — el backend los calcula
}
```

El backend responde con la referencia ya firmada, el importe calculado autoritativamente y la fecha_expiracion que él determinó.

Variantes que el back debe garantizar

- Un descuento no puede pertenecer a un cargo que no está en billing.cargos -> rechazar con 400
- En la consulta de los cargos solo regresar los ids descuentos, multas, totales para la visualización del front
- Solo_aplica_el_descuento_mayor si el tipo_de_cargo lo tiene activo, solo el descuento de mayor saldo_pendiente es válido, el resto se ignora aunque vengan en la lista
- Aplica_por_periodo, un descuento con este flag solo procede si el servidor confirma que todos los periodos del grupo están en billing.cargos.
- Descuento inactivo, estados_globales=5, cancelado=true o saldado=true → ignorar aunque venga en la lista, no retornar error (evita fuga de información sobre el estado de descuentos).
- descuentos_especiales, sólo válidos si estados_globales=4; cualquier otro estado los descalifica.
- El importe de la referencia debe ser la suma del adeudo_total recalculado por el servidor, nunca el valor enviado por el cliente.

RoadMap

Sprint 1

Seguridad crítica y funcionalidad rota

Estado	Actividad
☐ Sin completar	[FRONT] Eliminar log de NETPAY_PRIVATE_KEY en pagos.ts:L925
☐ Sin completar	[FRONT] Eliminar log del body de registro en auth.ts:L224–225
☐ Sin completar	[FRONT] Agregar Authorization header en actualizarMisDatos
☐ Sin completar	[FRONT] clearTransactionData: setItem → removeItem en storage.ts
☐ Sin completar	[FRONT] pnpm generate:types (resolver cast any en portal-settings.ts)
☐ Sin completar	[FRONT] Promise.all en getPortalSettings
☐ Sin completar	[FRONT] postMessage('*') → postMessage(APP_URL) en NetPayProvider.tsx (L54 y L69)
☐ Sin completar	[BACK] Verificar descuentos del propio contribuyente, no uno ajeno
☐ Sin completar	[BACK] Actualizar el contrato del API para que el frontend envíe únicamente la intención de pago (cargos, descuentos y datos del contribuyente), eliminando el envío de importes y fechas de expiración.
☐ Sin completar	[BACK] Migrar toda la lógica de cálculo al backend, incluyendo adeudos, recargos, actualizaciones, gastos, multas, accesorios, descuentos e importe total de la referencia.
☐ Sin completar	[BACK] Centralizar las validaciones de negocio en el servidor, verificando la elegibilidad de cargos y descuentos, estados, reglas de periodos y restricciones como solo_aplica_el_descuento_mayor.
☐ Sin completar	[BACK] Calcular en el backend la fecha de expiración de la referencia conforme a las reglas del padrón y calendario de días hábiles, ignorando cualquier valor enviado por el cliente.
☐ Sin completar	[BACK] Modificar la respuesta del servicio para devolver el importe final, la referencia

	de pago y la fecha de expiración calculados por el servidor.
☐ Sin completar	[BACK] Realizar pruebas funcionales y de seguridad para verificar que el backend ignore cualquier dato manipulable enviado por el cliente y que todas las reglas se apliquen correctamente.

Sprint 2

Headers de seguridad, observabilidad, UX crítica, Turnstile

Estado	Actividad
☐ Sin completar	[FRONT] Headers HTTP
☐ Sin completar	[AMBOS] SRI NetPay + validación scriptUrl
☐ Sin completar	[FRONT] Revalidar monto
☐ Sin completar	[AMBOS] Cloudflare Turnstile
☐ Sin completar	[AMBOS] Integrar Sentry + error.tsx
☐ Sin completar	[FRONT] Conectar calcularFechaExpiracionInteligente
☐ Sin completar	[AMBOS] Centralizar ENTIDAD_ID
☐ Sin completar	[AMBOS] Logging estructurado
☐ Sin completar	[FRONT] Corregir handleReset

Sprint 3

Funcionalidades incompletas y rendimiento

Estado	Actividad
☐ Sin completar	[AMBOS] DobleInputForm
☐ Sin completar	[AMBOS] PredioInputForm
☐ Sin completar	[AMBOS] Normateca
☐ Sin completar	[FRONT] Promise.allSettled
☐ Sin completar	[FRONT] ServiceRequestError
☐ Sin completar	[FRONT] Evaluar ISR
☐ Sin completar	[FRONT] Auditar auth

Sprint 4

Accesibilidad, refinamiento y deuda técnica

Estado	Actividad
☐ Sin completar	[AMBOS] Correlation IDs
☐ Sin completar	[FRONT] Auditoría PaymentSettings
☐ Sin completar	[BACK] login_attempts + logout
☐ Sin completar	[FRONT] MFA Payload
☐ Sin completar	[FRONT] Contraste WCAG
☐ Sin completar	[FRONT] role=alert
☐ Sin completar	[FRONT] Eliminar duplicados
☐ Sin completar	[FRONT] Eliminar fetchCabecerasTramitePublic
☐ Sin completar	[FRONT] Retry registrarVisita

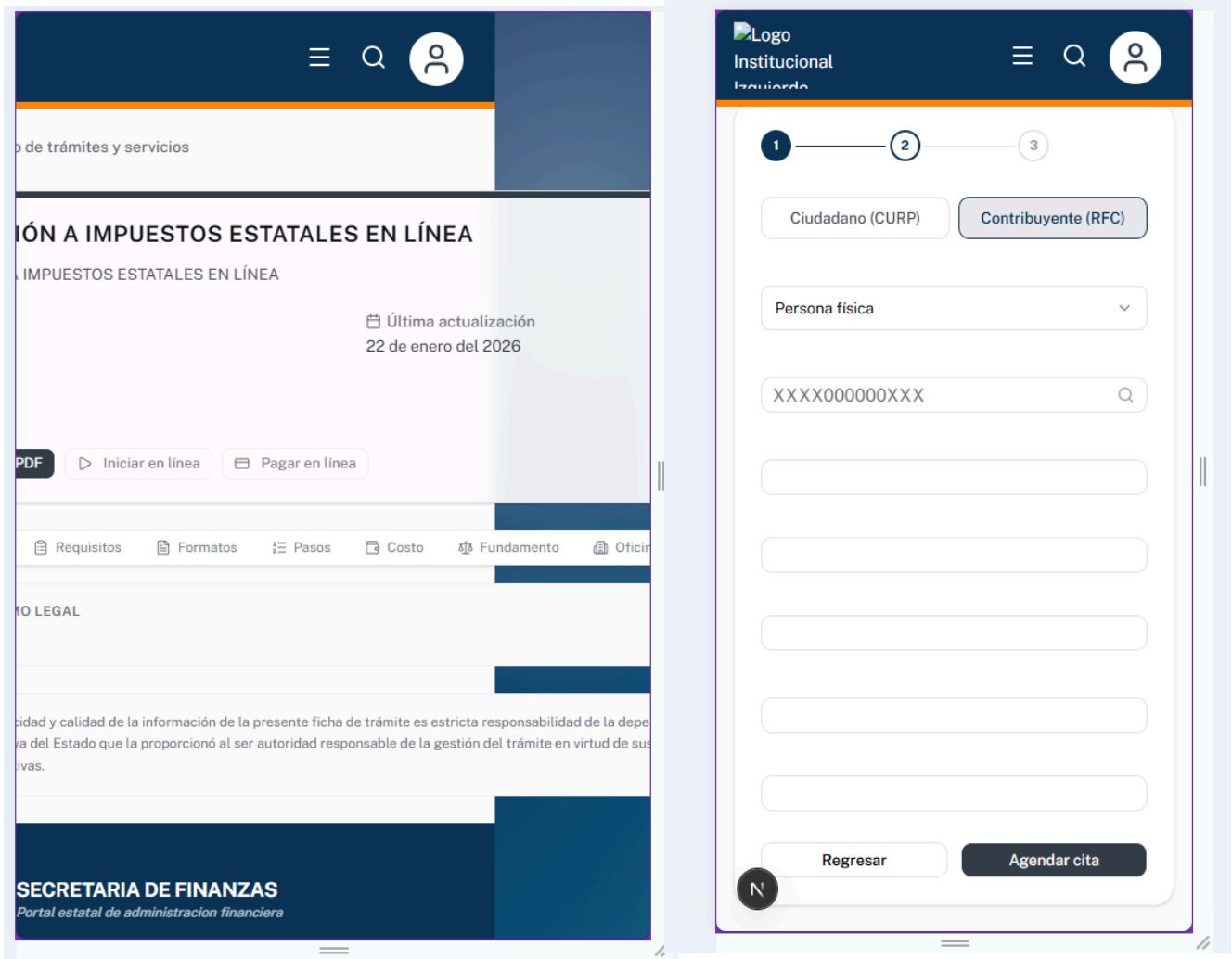
Pendientes

Módulo	Estado actual	Funcionalidad que falta
Quejas y Denuncias	100% simulado — consume <code>QUEJAS MOCK DATA</code> hardcodedo; no hay <code>actions/quejas.ts</code> ni endpoint real	Backend real de captura/consulta de quejas, y formulario para que el ciudadano presente una queja o denuncia nueva (no existe ningún formulario de alta)
Mi Buzón	Placeholder explícito: "La mensajería oficial está en desarrollo"	Mensajería oficial ciudadano ↔ dependencia
Pago en línea con tarjeta (flujo alternativo, clasificador "03")	Botón "Pagar en línea (NetPay)" solo genera la orden/PDF, igual que "Generar orden"	Invocar el SDK de NetPay y tokenizar la tarjeta realmente (el flujo principal de <code>consulta-adeudos</code> sí lo hace bien)
Búsqueda de contribuyente (servicios registrales, clasificador "02")	<code>DobleInputForm.tsx</code> devuelve una <code>mockResponse</code> fija ("Juan Pérez García") en vez de llamar al backend	Búsqueda real por RFC/CURP contra el servicio registral
Búsqueda de predio (servicios catastrales, clasificador "02")	<code>PredioInputForm.tsx</code> devuelve datos mock	Búsqueda real por clave catastral
Selección de cargos (clasificador "02")	<code>FormularioPago02Section.tsx</code> es solo un placeholder visual	Selección real de conceptos/cargos a pagar; sin esto el pago no puede completarse en este sub-flujo
Vencimiento inteligente de orden de pago	Existe la función correcta (<code>calcularFechaExpiracionInteligente()</code>) pero nunca se invoca; se usa un +3 días ingenuo que ignora fines de semana/festivos	Que el vencimiento real de la orden respete días inhábiles (como ya hace el módulo de Citas)

"Iniciar en línea" / "Pagar en línea" en detalle de trámite	Botones deshabilitados de forma permanente, sin bandera de configuración	Iniciar trámite o pagar en línea directamente desde el detalle del trámite
Descarga de PDF y cancelación de declaración	Ya implementado en backend (<code>fetchDeclaracionPdf</code> , <code>cancelarDeclaracion</code>) pero sin botón en la UI de "Mis Declaraciones"	Exponer esas acciones al ciudadano en la interfaz
Control de acceso por estado de padrón (Mi Cuenta)	El estado real (<code>user.estado_del_padron</code>) está comentado y sustituido por la constante 2 (Activo)	Que las secciones (Mis Establecimientos, Obligaciones, Pagos, Constancias, Declaraciones, Buzón, Vehículos) se habiliten solo según el estado real del padrón
RBAC de la colección Media	Solo define <code>access.read</code> ; <code>create/update/delete</code> quedan con el valor por defecto de Payload (público) y no está en la matriz de permisos de Roles	Definir permisos explícitos y agregarla a <code>managedCollections</code>
Declaraciones	Solo está en las rutas de pagos de impuestos	Se presenta el módulo más no tiene funcionalidad de crear una declaración, mostrar sus declaraciones etc

Pantallas

Diseño responsivo en los trámites, en móvil no se adapta debidamente
Cambios entre módulos no hay un indicador de cargado. hace el cambio “brusco”



Referencia:

[OWASP Top 10:2025 – Introduction](https://owasp.org/Top10/2025/0x00_2025-Introduction/). (2025). *OWASP Top 10:2025 – Introduction*.

https://owasp.org/Top10/2025/0x00_2025-Introduction/